📌 Sprint 2 补充交付物 —— Git Flow 分支治理 & Mock 前端闭环

📍 项目地址

https://github.com/handsome1qd/V1_For_Now

🧩 一、Git Flow 分支治理规范（团队提代码姿势）

🎯 目标

通过规范 Git 分支管理流程，实现：

- 防止污染主干代码
- 保证多人协作稳定性
- 降低代码冲突风险
- 强制代码评审（PR Review）

📜 核心规则（强制执行）

❗ 规则1：禁止直接提交主干

- ❌ 不允许 `git push origin main`
- ❌ 不允许 `git push origin develop`

👉 原因：
主干必须保持稳定、可运行状态 :contentReference[oaicite:0]{index=0}

❗ 规则2：必须使用 feature 分支开发

- 命名规范：
  feature/功能名

示例：
feature/combat
feature/login
feature/user-system

👉 原因：
Feature Branch Workflow 要求所有开发在独立分支进行 :contentReference[oaicite:1]{index=1}

❗ 规则3：PR 前必须“反向同步主干”

👉 核心考点（最重要）

在提交 PR 前必须执行：
git checkout develop
git pull origin develop

git checkout feature/xxx
git merge develop

👉 作用：

- 提前解决冲突
- 保证不会破坏他人代码
- 保证 PR 可顺利合并

👉 原理：
Feature 分支需要持续同步 develop 分支以减少冲突 :contentReference[oaicite:2]{index=2}

❗ 规则4：必须走 PR + Review 流程

流程：

1. push feature 分支

2. 发起 Pull Request
3. 至少 1 人 Approve
4. 才允许 merge

👉 原因：
PR 提供代码评审与讨论机制 :contentReference[oaicite:3]{index=3}

🔄 二、标准 Git Flow 操作流程（必须执行两次）

✅ 第一次完整流程（示例）

🟢 Step1：创建分支

```
git checkout develop
git pull origin develop
git checkout -b feature/test1
```

🟢 Step2：开发 & 提交

```
git add .
git commit -m "feat: add test1 feature"
```

🟢 Step3：同步主干（关键步骤）

```
git checkout develop
git pull origin develop

git checkout feature/test1
git merge develop
```

🟢 Step4：推送分支

git push origin feature/test1

🟢 Step5：发起 PR（GitHub）

- 目标分支：develop
- 标题：feat: test1
- 描述：实现xxx功能

🟢 Step6：Code Review

- 队友点击 Approve
- 可评论修改

🟢 Step7：合并 PR

- Merge into develop
- 删除 feature 分支

✅ 第二次流程（示例）

feature/test2

👉 重复上述步骤（必须截图证明）

📸 截图要求（必须提交）

1. 分支创建截图

2. commit记录截图
3. PR页面截图
4. Approve截图
5. Merge截图

📊 三、Git Flow 规范总结

项目	是否允许
直接 push main	❌
直接 push develop	❌
使用 feature 分支	✅
PR 审核	✅
同步主干	✅

🧪 四、Mock 环境 —— Prism 前端闭环

🎯 目标

在后端未完成情况下：

- 前端先跑通流程
- 模拟 API 数据
- 实现页面交互闭环

🧰 工具说明

使用工具：

👉 **Prism（Mock Server）**

作用：

- 根据 OpenAPI / YAML 文件
- 自动返回模拟数据

📄 Step1：编写接口合同（YAML）

示例： `api.yaml`

```yaml
openapi: 3.0.0
info:
  title: Mock API
  version: 1.0.0

paths:
  /login:
    get:
      summary: 用户登录
      responses:
        '200':
          description: 成功
          content:
            application/json:
              example:
                code: 200
                message: success
                data:
                  username: test_user
```